

DeGro: Target Determinacy Verification and Grounded Repair for Executable Mathematical Reasoning

Anonymous Author(s)
Submission Id: ANONYMOUS

Abstract

Executable mathematical reasoning can succeed while encoding an underconstrained or mistranslated problem. Execution checks catch implementation failures but cannot establish whether the encoded constraints determine the requested quantity. We present *DeGro*, a neuro-symbolic framework for target determinacy diagnosis and grounded repair. *DeGro* asks whether two feasible assignments can disagree on the target; when they do, it accepts only source-supported constraints that pass provenance and formal checks, and otherwise abstains. We evaluate *DeGro* conditional on an available structured specification, using controlled paired settings and human-verified studies of naturally occurring formalization errors. Across the MIRA-Math and DRAW-Paired evaluations, *DeGro* generally improves repair-or-abstain accuracy over grounded self-review across multiple language models. On natural errors, it improves accuracy for supported MATH-500 cases, while ALG514 reveals answer-compatible formalization errors that ordinary answer matching can miss. Ambiguity detection remains substantially easier than repair, and explicit counterexample witnesses provide no measurable benefit beyond the determinacy verdict. Target determinacy is therefore an auditable guardrail for executable reasoning, not a guarantee of source-to-formal faithfulness or an end-to-end evaluation of code generation.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Intelligent agents**; *Natural language processing*.

Keywords

neuro-symbolic reasoning, mathematical agents, verification, counterexamples, abstention

ACM Reference Format:

Anonymous Author(s). 2027. DeGro: Target Determinacy Verification and Grounded Repair for Executable Mathematical Reasoning. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 8 pages.

1 Introduction

Large language models increasingly delegate mathematical computation to programs and symbolic solvers. Program-aided reasoning reduces arithmetic errors, while systems such as SymCode generate self-contained symbolic programs and repair execution failures [5, 15]. Successful execution, however, does not establish



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

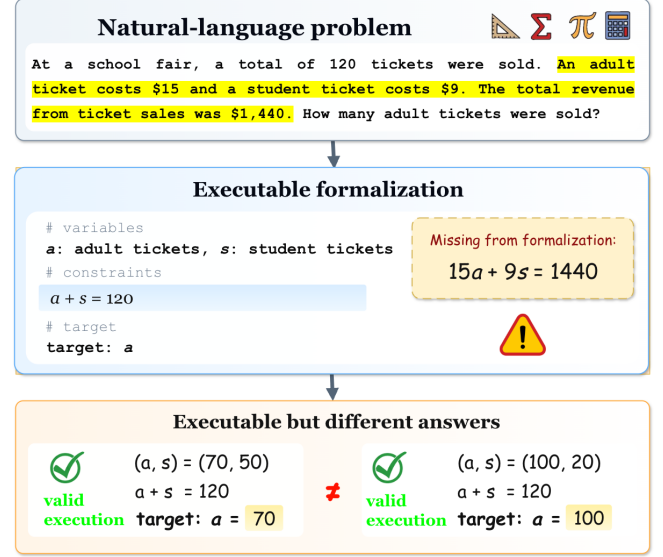


Figure 1: An executable but target-ambiguous formalization caused by an omitted constraint.

that the encoded constraints determine the requested answer. A program may run and return a value even when its formalization admits several values of the target.

Figure 1 illustrates the failure mode. The source states both the total number of tickets and the total revenue, but the executable formalization retains only $a + s = 120$. Both $(a, s) = (60, 60)$ and $(100, 20)$ satisfy this incomplete model while giving different values of the requested number of adult tickets. We call this condition *target ambiguity*: feasible assignments disagree on the requested quantity. It concerns the target rather than uniqueness of the entire assignment, since freedom in other variables need not affect the answer.

Detecting ambiguity does not establish how to resolve it. The same model faithfully represents the shorter source “Ten tasks are divided between A and B. How many does A receive?” For the complete source, adding $a = b + 2$ recovers an omitted relation; for the shorter source, the same addition fabricates information. The formalization and solver verdict are identical, but the correct action changes with source support. A repair policy must therefore distinguish evidence that the target is ambiguous from evidence that a particular constraint is justified.

We propose *DeGro*, a neuro-symbolic framework that separates these two decisions. It first constructs two renamed copies of a generated specification and asks whether both can satisfy the constraints while disagreeing on the target. A satisfying query provides

auditable evidence of ambiguity. The agent then proposes a constraint with supporting source text, and acceptance checks require consistency, non-redundancy, and restored target determinacy. If no admissible repair is found, the agent abstains. The solver guarantee concerns the encoded specification: even a determinate target can be wrong when the target or constraints were mistranslated.

We evaluate this framework at two complementary levels. A controlled paired benchmark holds the formalization and diagnostic fixed while varying source support, testing whether the policy repairs a supported omission and abstains on its underspecified counterpart. Human-verified natural-error studies then examine representation coverage, ambiguity detection, and successful grounded repair separately. The controlled results show improved repair-or-abstain decisions, while the natural errors expose a substantial gap between detecting ambiguity and repairing its cause. We also test whether concrete witness assignments improve decisions beyond the solver verdict. In all experiments, DeGro is evaluated conditional on the source problem and an available candidate MODELSPec; we do not evaluate executable-code generation or a code-to-MODELSPec adapter, and therefore make no end-to-end improvement claim over SymCode.

Our research questions are:

(RQ1) Decision quality and tradeoffs. How do target-determinacy diagnosis and source grounding affect repair-or-abstain accuracy and the tradeoff between supported repair and unsupported additions?

(RQ2) Paired source sensitivity. When the formalization and diagnostic are held fixed, can an agent both repair the source-supported omission and abstain on the underspecified member of the same pair?

(RQ3) Practical bottlenecks. How do representation coverage, ambiguity detection, and grounded repair limit effectiveness on natural errors, and do concrete witnesses improve controlled repair decisions beyond the verdict?

Takeaway

DeGro checks whether the encoded constraints determine the requested target, requires cited support and formal validation for repairs, and otherwise abstains.

Our main contributions are:

(1) A target-determinacy-guided repair-or-abstain framework. We introduce DeGro, which combines two-copy target-determinacy diagnosis with source-grounded repair, deterministic acceptance gates, and explicit abstention (RQ1).

(2) Controlled and natural-source paired benchmarks. We construct matched cases that hold the formalization fixed while varying whether the source supports the missing constraint. Alongside MIRA-Math, DRAW-Paired contributes 300 author-reviewed test pairs established from the beginning from natural algebra word

problems, isolating supported repair from unsupported completion (RQ2).

(3) A multi-stage empirical audit of executable reasoning. We combine controlled and human-verified natural-error evaluations to separate representation coverage, ambiguity detection, and grounded repair; we additionally test whether concrete witnesses improve decisions (RQ3).

The formal guarantee concerns F ; the empirical answers concern model and policy behavior.

2 Related Work

2.1 Executable Mathematical Reasoning

PAL and Program-of-Thoughts separate language-based reasoning from program execution to reduce computational errors [8, 15]. MathCoder and ToRA integrate code and tool calls into mathematical problem solving [18, 35], while faithful chain-of-thought connects generated rationales to executable symbolic procedures [25]. SymCode is particularly close to our setting: it represents mathematical reasoning as a SymPy program and uses execution feedback to repair programmatic failures [5]. These approaches motivate an additional check after successful execution. A generated artifact may return a value even though its constraints admit other values of the requested quantity. DeGro checks this property of the encoded feasible region and routes ambiguous specifications to grounded repair or abstention. Integration with such systems requires an adapter that exposes their encoded variables, constraints, and target as a MODELSPec; evaluating that adapter and the resulting full pipeline is outside the present study.

Outcome verification and process supervision assess candidate solutions or intermediate reasoning steps [10, 24, 33]. Self-Refine uses model-generated feedback, and CRITIC incorporates external tools into revision [17, 26]; evidence on intrinsic self-correction shows the limits of revision without reliable external feedback [20]. DeGro contributes a specific solver observation: two feasible assignments disagree on the target. This observation establishes ambiguity under the representation, independently of the predicted answer, but does not identify which source constraint should be added.

2.2 Symbolic Diagnosis and Relational Verification

Recent work addresses semantic failures that survive execution. ReLoop tests optimization formulations through solver-based parameter perturbations [22]. SymDiag translates reasoning steps into constraints and uses satisfiability and entailment checks to localize defects, with a Self-Auditor to distinguish translation errors from reasoning errors [11]. Minimal-core-guided repair instead returns an unsatisfiable subset of generated constraints as evidence for revising inconsistent translations [23, 30]. DeGro focuses on a complementary condition: the specification is satisfiable, but its feasible assignments disagree on the requested target. Its diagnosis concerns the specification as a whole rather than the validity of individual derivation steps, and its additive repair policy requires support from the original source.

The two-copy construction builds on self-composition and product programs, which reduce relational verification to queries over

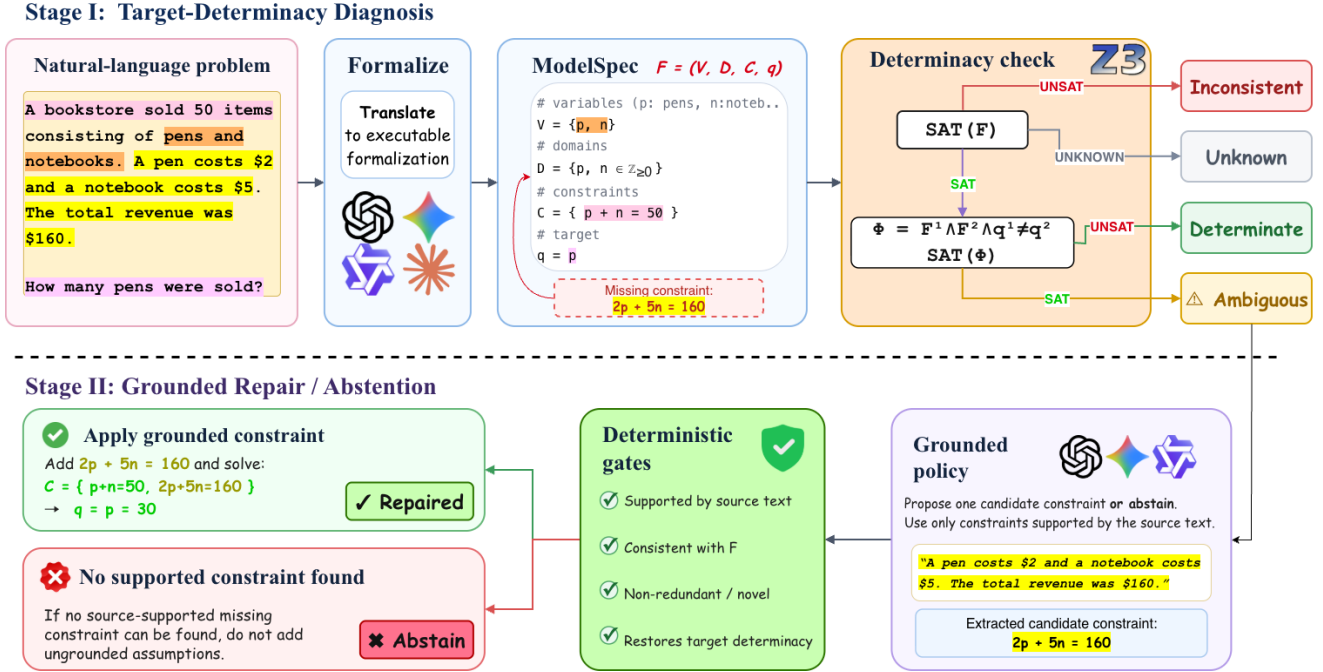


Figure 2: Overview of DeGro’s target-determinacy diagnosis and grounded-repair pipeline.

renamed copies [6, 7, 32]. DeGro instantiates this established technique as target agreement across feasible assignments. Agreement on the target is weaker than uniqueness of the full assignment: nuisance variables may remain free without making the answer ambiguous. Counterexample-guided refinement and synthesis likewise use solver evidence to guide changes [9, 31]. Here, however, a counterexample refutes target determinacy without supplying a semantically justified edit. Our contribution is the integration of this diagnosis with a source-sensitive repair policy and its evaluation, rather than a new relational verification primitive.

2.3 Grounded Repair, Abstention, and Faithfulness

Classical diagnosis, abductive inference, and formal model repair provide foundations for identifying defects and modifying specifications to satisfy a desired property [3, 13, 29]. LLM-based repair extends this direction to declarative formal specifications [2]. Restoring determinacy alone is insufficient in our setting, since a guessed equality can force a unique target. DeGro therefore requests a source-supported constraint and checks consistency, non-redundancy, and restored determinacy before acceptance. These formal gates verify the effect of the edit; source support remains a separate semantic requirement.

Selective prediction studies when to withhold an answer under a risk–coverage tradeoff [14, 16], and semantic uncertainty estimates uncertainty in language-model outputs [21]. DeGro bases its repair-or-abstain decision on two distinct questions: whether the encoded target is determinate and whether the source supports a constraint that resolves ambiguity. MIRA-Math is directly relevant

because it evaluates requesting and integrating necessary missing information [1]. We adapt its validated latent states into matched sources with identical formalizations but different support for repair, isolating the decision to recover an omitted relation or abstain from unsupported completion.

Autoformalization also separates formal validity from preservation of natural-language meaning [4, 36]. DeGro’s guarantee is similarly representation-relative: a determinate specification can still contain a wrong target or an unsupported constraint. Our paired and human-verified natural-error evaluations therefore distinguish determinacy diagnosis, grounded repair, and answer accuracy rather than treating a solver verdict as proof of source-to-formal faithfulness.

3 Methodology

DeGro checks whether a candidate mathematical specification determines the requested target. Its evaluated interface is the pair (P, F) : the source problem P and an available `ModelSpec` F . The upstream mechanism that produced F may be a direct autoformalizer, a program adapter, or another reasoning system; it is not part of the controlled intervention evaluated here. Stage I diagnoses target ambiguity using a symbolic solver. Stage II attempts to resolve an ambiguous target by adding a constraint with source provenance, or abstains if no candidate is accepted. Figure 2 summarizes this component boundary.

3.1 Problem Setup and Target Determinacy

Given a natural-language problem P and a candidate specification $F = (V, D, C, q)$, V is the set of variables, D their domains, C the constraints, and $q(V)$ the requested expression. Its feasible assignments are

$$\text{Sol}(F) = \{m \mid m \models D \wedge C\}. \quad (1)$$

For a satisfiable F , the target is *determinate* when every feasible assignment gives the same target value:

$$\text{Det}(F, q) \iff \forall m_1, m_2 \in \text{Sol}(F), \quad q(m_1) = q(m_2). \quad (2)$$

Target determinacy does not require a unique assignment to all variables. For example, $x = 5, y \geq 0$ fixes the target $q = x$ even though y remains free. An inconsistent specification is reported separately and never receives a determinacy certificate.

The specification is stored as a typed **MODELSPEC**: a structured record of variable sorts and bounds, source-linked constraints, and the target. Computational and verification backends consume this same representation, avoiding discrepancies between independently generated programs. The supported fragment includes integer, real, and Boolean variables, arithmetic equalities and inequalities, finite domains, and selected modular constraints.

The certificate concerns F , not whether F correctly represents P . Wrong or extra constraints can fix a target while mistranslating the source. DeGro therefore repairs only ambiguous specifications by adding constraints; correcting a determinate but incorrect representation is outside its scope.

3.2 Stage I: Target-Determinacy Diagnosis

DeGro first checks whether $D \wedge C$ is satisfiable. An unsatisfiable result yields **INCONSISTENT**; a timeout or inconclusive solver result yields **UNKNOWN**. If the specification is satisfiable, DeGro constructs two copies with disjoint variable names and asks whether they can produce different target values:

$$\Phi(F, q) = F^{(1)} \wedge F^{(2)} \wedge q(V^{(1)}) \neq q(V^{(2)}). \quad (3)$$

If Φ is satisfiable, its projections form a *target-divergent witness pair*, and the status is **AMBIGUOUS**. If Φ is unsatisfiable, the status is **DETERMINATE**, since the initial check has already established that F has a solution. Unsupported representations yield **UNSUPPORTED** before solver verification. Only **AMBIGUOUS** enters Stage II.

Algorithm 1 Target-determinacy verification

Require: Typed specification $F = (V, D, C, q)$
Ensure: Status and optional witness pair

```

1:  $r_0 \leftarrow \text{CHECKSAT}(D \wedge C)$ 
2: if  $r_0 = \text{UNSAT}$  return INCONSISTENT
3: if  $r_0 \neq \text{SAT}$  return UNKNOWN
4:  $\Phi \leftarrow F^{(1)} \wedge F^{(2)} \wedge (q^{(1)} \neq q^{(2)})$  using disjoint copies
5:  $(r_1, M) \leftarrow \text{CHECKSAT}(\Phi)$ 
6: if  $r_1 = \text{UNSAT}$  return DETERMINATE
7: if  $r_1 \neq \text{SAT}$  return UNKNOWN
8: return AMBIGUOUS,  $\text{PROJECTWITNESSES}(M)$ 
```

Soundness. Assuming semantics-preserving compilation and a sound solver for the supported theory, $\text{SAT}(F)$ together with $\text{UNSAT}(\Phi)$ establishes target determinacy: any two feasible assignments with different targets would satisfy Eq. (3), contradicting unsatisfiability. Conversely, a satisfying assignment to Φ

supplies two feasible witnesses with different targets. Timeout and **UNKNOWN** provide no certificate. The implementation uses a restricted expression compiler and separate variable maps for the two Z3 copies [12].

3.3 Stage II: Grounded Repair or Abstention

The verifier retains the target-divergent witness pair as a certificate, but the default repair prompt receives only the source problem, the encoded constraints and target, and the typed **AMBIGUOUS** verdict. It returns either **ADD-CONSTRAINT** with an exact supporting source span (or a permitted domain rule), or **ABSTAIN**. It receives neither the hidden gold target nor the missing gold constraint. An additional ablation exposes the concrete witnesses to test whether their values help beyond the verdict; they do not improve decisions on the present benchmark. In either variant, the model must recover a relation from the source rather than select a witness as correct.

Acceptance gate. For a proposed constraint c and a frozen set $\mathcal{R}$ of permitted domain-semantics rules, the deterministic gate accepts c only when

$$\text{Accept}(c) \iff \text{Compiles}(c) \wedge \text{Provenance}(c, P, \mathcal{R}) \wedge \text{SAT}(F \wedge c) \wedge \neg(F \models c) \wedge \text{Det}(F \wedge c, q). \quad (4)$$

These checks require that the candidate compile, cite a source span occurring in P or a rule in $\mathcal{R}$, preserve consistency, add a non-redundant restriction, and restore target determinacy. Compilation failures and inconclusive checks do not permit acceptance.

Provenance is an auditable check, not a proof that the constraint correctly expresses the cited text. Passing the gate therefore establishes valid provenance and determinacy under the repaired specification, rather than semantic correctness with respect to P . Semantic source support and functional correctness are evaluated separately for repair credit; an incorrect translation or unsupported target pin does not earn credit merely by passing the gate.

Bounded repair. DeGro allows at most two proposal attempts. The first accepted constraint is committed and terminates the loop. If the first proposal is rejected, the second attempt receives the rejection reason and operates on the unchanged F . Rejected constraints are not accumulated: each candidate must restore determinacy on its own, and a supported but insufficient addition is rejected. If the model abstains or neither candidate passes, the outcome is **NO-ADMISSIBLE-REPAIR-FOUND**, and F is preserved.

Failure to find an admissible repair does not prove that the source is underspecified: the model may have missed a relation already present in the source.

Controller policy. Table 1 maps verifier states to agent actions. A determinate result permits an answer scoped to the current specification, including an accepted repair when present. In interactive use, failure to find a repair can trigger clarification or abstention; the benchmark scores abstention because no clarification channel is available. Inconsistent, unknown, and unsupported states require distinct fallbacks rather than being treated as ambiguous inputs. The parser, compiler, solver, and gate form the trusted boundary; the language model only proposes specifications and repairs. Logs retain solver verdicts, witnesses, proposals, cited support, and rejection reasons for auditing.

Table 1: Controller semantics for DeGro integration.

Verifier state	Evidence	Controller action
Determinate	F sat, Φ unsat	answer with scope
Ambiguous	targets disagree	grounded repair
Inconsistent	F unsat	revise encoding
Unknown	solver inconclusive	fallback/escalate
Unsupported	outside fragment	alternate tool
No repair found	no accepted candidate	ask or abstain

4 Experiments and Results

4.1 Setup

4.1.1 Evaluation Scope. We evaluate repair policies along three complementary dimensions: (i) *diagnostic guidance*, whether a solver-produced target-ambiguity verdict improves revision; (ii) *source grounding*, whether exact source evidence prevents unsupported additions; and (iii) *repair-or-abstain behavior*, whether a policy repairs a supported omission while withholding an edit when the source is genuinely underspecified. The first two dimensions address RQ1, whereas repair-or-abstain behavior on matched sources directly addresses RQ2.

4.1.2 Baselines. To evaluate the effectiveness of DeGro, we compare it against representative repair policies that use weaker diagnostic or grounding signals. All methods receive the same source, current constraints, and target; begin from the same initial formalization; and use a two-attempt repair budget. Differences therefore reflect the feedback and repair policy rather than initial-generation quality. We consider the following baselines:

- **Self-review.** The model revises the formalization using the source and current artifact, but receives no explicit solver diagnosis and is not required to ground an added constraint in the source.
- **Grounded self-review.** The model follows the same exact-span grounding rule used by DeGro, so every introduced constraint must cite supporting source text. It receives no solver feedback, isolating grounding without determinacy guidance.
- **Determinacy-guided repair.** The model receives the typed AMBIGUOUS verdict but no exact-span grounding requirement. This baseline isolates the effect of determinacy feedback without source-grounded control.

4.1.3 Models. We evaluate all four policies with GPT-OSS 20B and GPT-OSS 120B [28], and Nemotron 3 Nano 30B [27]. All models use the same cases, frozen prompt, temperature 1.0, and repair budget.

4.1.4 Metrics. Accuracy. Our primary metric counts a decision as correct when the method produces a grounded, functionally correct repair for an omission case or abstains on an underspecified case. Because each controlled pair contains one case of each type, accuracy is equivalently computed as

$$\text{Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}.$$

Paired benchmark composition

660 pairs across two benchmarks

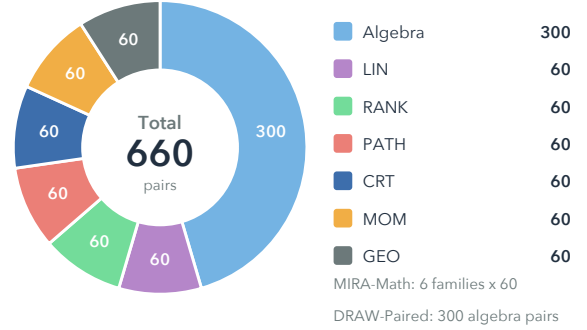


Figure 3: Paired benchmark composition. DRAW-Paired contributes 300 pairs constructed from natural algebra word-problem sources; MIRA-Math contributes six 60-pair families: linear separation (LIN), rank-deficient systems (RANK), path sums (PATH), Chinese-remainder reconstruction (CRT), moment constraints (MOM), and geometry (GEO). Every pair yields one OMISSION and one UNDERSPECIFIED case.

Recall and specificity. Recall measures the proportion of omission cases repaired correctly, whereas specificity measures the proportion of underspecified cases on which the method correctly abstains. We omit the false-positive rate because it is exactly $1 - \text{Specificity}$. **Unsupported repair rate.** URR is the share of proposed additions lacking source support and is reported with recall to account for abstention. Semantic, functional, and grounded outcomes remain separate. Accuracy comparisons use exact paired McNemar tests with Holm correction across models; percentage-point intervals use 10,000 pair-cluster bootstrap resamples.

4.2 Datasets

We evaluate on two matched-pair benchmarks: 360 MIRA-Math pairs across six controlled mathematical families and 300 DRAW-Paired test pairs derived from natural algebra word problems. Figure 3 summarizes their composition.

Both benchmarks use the same construction. Removing one target-critical constraint yields a satisfiable but target-ambiguous specification shared by both members of a pair. The OMISSION source retains the supporting fact and should be repaired; the UNDERSPECIFIED source removes it and should trigger abstention. In MIRA-Math, the missing fact is provided by the dataset and every retained pair passes satisfiability, ambiguity, and restored-determinacy checks [1]. DRAW-Paired applies this design to 300 author-reviewed test pairs constructed from natural algebra word problems in DRAW-1K [34] from the beginning. All supporting spans and deletions were author-reviewed. Full split, filtering, and provenance details are reported in the supplement.

Table 2: Main paired experiments on MIRA-360 (360 pairs, 720 cases across six families) and DRAW-Paired (300 author-reviewed test pairs, 600 cases). Values are percentages. Accuracy is primary; recall and specificity decompose repair and abstention, and URR measures unsupported additions among proposed additions. Blue rows denote DeGro; bold accuracy marks the best policy within each model and dataset.

Model	Method	MIRA-Math				DRAW-Paired			
		Acc.↑	Recall↑	Spec.↑	URR↓	Acc.↑	Recall↑	Spec.↑	URR↓
GPT-OSS 20B	Self-review	90.6	93.1	88.1	14.5	73.8	66.7	81.0	43.3
	Grounded self-review	91.7	94.7	88.6	11.2	73.3	62.3	84.3	43.8
	Determinacy-guided	92.6	88.1	97.2	7.9	73.7	57.0	90.3	46.1
	DeGro (ours)	96.3	93.3	99.2	2.9	79.2	66.7	91.7	36.3
GPT-OSS 120B	Self-review	78.6	99.4	57.8	29.9	72.0	68.3	75.7	44.1
	Grounded self-review	89.6	96.7	82.5	15.5	69.0	54.0	84.0	51.1
	Determinacy-guided	92.4	98.6	86.1	13.4	77.8	69.3	86.3	37.5
	DeGro (ours)	96.1	97.8	94.4	6.4	77.8	63.7	92.0	38.8
Nemotron Nano 30B	Self-review	83.9	76.1	91.7	25.5	63.5	44.7	82.3	59.0
	Grounded self-review	86.7	75.0	98.3	4.6	60.7	35.7	85.7	61.9
	Determinacy-guided	72.9	53.9	91.9	27.3	63.3	34.7	92.0	55.4
	DeGro (ours)	89.6	79.4	99.7	1.0	70.8	48.3	93.3	44.4

4.3 Main Paired Results

MIRA-Math. Table 2 reports the MIRA-Math aggregate. DeGro achieves the highest accuracy for all three models, improving over grounded self-review by 2.9–6.5 points; all comparisons remain significant after Holm correction. Full confidence intervals and tests appear in the supplement.

Grounding trade-off. DeGro lowers URR and raises specificity for all three models while broadly preserving recall. Thus the reduction in unsupported additions does not merely reflect universal abstention.

DRAW-Paired. On DRAW-Paired (300 test pairs evaluated from the beginning), DeGro improves accuracy over grounded self-review by 5.8–10.2 points across models, with all comparisons significant after Holm correction. For 120B, it ties determinacy-guided repair for the best accuracy while favoring specificity over recall. Full confidence intervals, tests, and discordance counts appear in the supplement.

5 Analysis

5.1 Natural-Error Analysis

We additionally evaluate naturally occurring formalization errors rather than injected omissions. GPT-OSS 20B generates one structured specification for each problem, and two reviewers verify the resulting semantic labels. We retain 292 conclusive MATH-500 cases and 503 ALG514 cases; Section 5.4 reports the excluded cases separately.

On MATH-500, 24 specifications contain a source-supported, target-critical omission. DeGro detects 21 of them (87.5% recall), but only one is repaired successfully end to end. It increases answer accuracy from 49.3% (144/292) to 53.8% (157/292), with 16 paired changes favoring DeGro and three favoring the structured baseline

(exact McNemar $p = .0044$). The low repair count shows that detecting ambiguity is easier than converting it into a correct grounded edit.

On ALG514 [34], 503 of 514 cases are supported by the current representation. Structured Solver and DeGro obtain 94.2% and 94.4% accuracy, respectively. Nine numerically acceptable outputs still contain a human-verified semantic error, showing that answer accuracy alone can hide representation mistakes. Overall, the natural-error study supports DeGro mainly as a diagnostic and routing mechanism; reliable automatic repair remains the main bottleneck.

5.2 Family-Level Analysis

For 20B, a bare verdict raises specificity to 98.0% while lowering recall to 87.7%; DeGro restores repairs, reaches 99.0% specificity, and reduces unsupported additions from 12.5% to 3.4% relative to grounded self-review. For GPT-OSS 120B, DeGro raises specificity from 79.7% to 93.3% and recall from 97.0% to 98.0%, while reducing unsupported additions from 17.3% to 7.0%. Family effects remain concentrated rather than uniform.

Table 3 explains the RQ1 tradeoff behind the aggregate scores. Generic review often inserts a plausible but unstated relation; DeGro blocks many such additions. The 20B rank-deficient family remains an adverse regime, whereas 120B improves across all six MIRA families and DRAW-Paired algebra. Lower URR therefore cannot by itself establish better grounded decision-making; recall, specificity, and family-specific losses must be read with it.

5.3 Witness Ablation

On the 100-pair development set, DeGro scores 98.5% accuracy versus 97.5% with witnesses (five versus three discordances favoring DeGro; Holm-adjusted $p = .727$). A minimal witness raises recall by six points over a bare verdict, also non-significantly ($p = .714$). Concrete assignments therefore add no measurable benefit here:

Table 3: Accuracy (%) by family for the six MIRA-Math families and DRAW-Paired algebra. GSR denotes grounded self-review.

Family	20B		120B	
	GSR	DeGro	GSR	DeGro
CRT reconstruction	100.0	99.2	99.2	100.0
Graph path sums	96.7	99.2	88.3	97.5
Linear separation	76.7	97.5	85.0	90.8
Moment constraints	88.3	97.5	84.2	95.0
Rank-deficient systems	94.2	87.5	90.0	95.0
Geometry	94.2	96.7	95.8	98.3
Algebra (DRAW-Paired)	73.3	79.2	69.0	77.8

Table 4: GPT-OSS 20B ablation on 100 development pairs (%). Concrete witnesses do not improve accuracy over DeGro without witnesses.

Method	Recall	Spec.	Acc.
Self-review	92	84	88
Bare verdict	87	97	92
Minimal witness	93	93	93
Target witnesses	86	94	90
Minimal witness + ground	97	99	98
DeGro	97	100	98.5
DeGro + witnesses	97	98	97.5

the typed verdict can be exposed by default and witnesses retained for audit or harder cases.

Every development instance contains one target-critical omission, so the determinacy verdict already localizes the relevant defect. Witness assignments add tokens but little useful information on this set; they may still help with harder specifications containing several omissions or many nuisance variables.

5.4 Coverage Analysis

To measure the representation bottleneck preceding verification, GPT-OSS 20B directly generated one shared candidate `MODEL_SPEC` for each MATH-500 problem [19]. No executable program or code-to-specification adapter was used. Of 500 specifications, 292 (58.4%) parsed, reached the solver, and produced a conclusive verdict; Table 5 accounts for every exclusion.

Coverage and natural-error performance answer different questions. Coverage measures whether a problem can be represented and decided by the current pipeline; natural-error analysis measures DeGro’s behavior after a usable specification exists. Thus the MATH-500 accuracy above is conditional on the 292 conclusive cases, while the remaining 208 cases expose representation, solver, or execution limitations. Reporting both prevents conditional performance from hiding 41.6% of the benchmark.

Table 5: Frozen MATH-500 cohort accounting for all 500 shared initial structured specifications.

Outcome	Count	Share (%)
Conclusive	292	58.4
Solver unsupported	112	22.4
Model unsupported	65	13.0
Call error	13	2.6
Inconsistent	9	1.8
Solver unknown	9	1.8
Total	500	100.0

6 Discussion and Limitations

DeGro’s main benefit is conservative decision-making: it reduces unsupported repairs by checking whether the target is determined and whether a proposed constraint is supported by the source. The paired design isolates this decision, but the controlled MIRA cases are templated and do not represent all errors found in natural mathematical language.

The certificate is relative to the generated specification. A wrong or extra constraint can still produce a unique but incorrect target, and the current solver does not cover many nonlinear, transcendental, or diagram-based problems. Natural-error results also show that high ambiguity-detection recall does not imply successful repair. For deployment, DeGro should therefore be used as a routing signal—answer, repair, clarify, or abstain—rather than as a general correctness guarantee.

7 Conclusion

DRAW-Paired extends the design to 300 author-reviewed natural algebra test pairs from the start. DeGro improves accuracy over grounded self-review across all three models, and ties verdict-only repair for GPT-OSS 120B, demonstrating that determinacy guidance effectively transfers to natural algebra word problems. On MIRA-360, DeGro improves accuracy over grounded self-review across all three models, with all comparisons surviving Holm correction. For RQ2, decisions depend on source support even when formalization and determinacy signal are identical. For RQ3, coverage and detection substantially exceed overall grounded-repair success on natural errors, while concrete witnesses add no measurable benefit on the single-omission development set. Target determinacy is a representation-relative routing condition, with grounding and coverage reported alongside decision accuracy. These findings characterize DeGro conditional on a candidate `MODEL_SPEC`; they do not establish an end-to-end improvement over SymCode or another executable-code generator.

References

- [1] Charbel Al Bateh and Samer Saab Jr. 2026. MIRA-Math: A Benchmark for Minimal Information Requesting and Mathematical Reasoning. *arXiv preprint arXiv:2607.07391* (2026).
- [2] Mohammad Alhanahnah, Md Rashedul Hasan, and Hamid Bagheri. 2024. An Empirical Evaluation of Pre-trained Large Language Models for Repairing Declarative Formal Specifications. *arXiv preprint arXiv:2404.11050* (2024).
- [3] Paul C. Attie, Kinan Dak Al Bab, and Mouhammad Sakr. 2017. Model and Program Repair via SAT Solving. *ACM Transactions on Embedded Computing Systems* 17, 2, Article 32 (2017). doi:10.1145/3147426
- [4] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. 2023. ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. *arXiv preprint arXiv:2302.12433* (2023).
- [5] Sina Bagheri Nezhad, Yao Li, and Ameeta Agrawal. 2026. SymCode: A Neurosymbolic Approach to Mathematical Reasoning via Verifiable Code Generation. In *Findings of the Association for Computational Linguistics: EACL 2026*. Association for Computational Linguistics, 1489–1503. doi:10.18653/v1/2026.findings-eacl.76
- [6] Gilles Barthe, Juan Manuel Crespo, and César Kunz. 2011. Relational Verification Using Product Programs. In *FM 2011: Formal Methods (Lecture Notes in Computer Science, Vol. 6664)*. 200–214. doi:10.1007/978-3-642-21437-0_17
- [7] Gilles Barthe, Pedro R. D’Argenio, and Tamara Rezk. 2004. Secure Information Flow by Self-Composition. In *17th IEEE Computer Security Foundations Workshop*. 100–114. doi:10.1109/CSFW.2004.1310735
- [8] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023).
- [9] Edmund M. Clarke, Orna Grumberg, Suresh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 1855)*. 154–169. doi:10.1007/10722167_15
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. In *arXiv preprint arXiv:2110.14168*.
- [11] Wenyao Cui, Huaping Zhang, Yongyi Huang, Qiuchi Li, Jian Xu, Cheng-Lin Liu, Chunxiao Gao, Juan Wang, and Baohua Zhang. 2026. SymDiag: Explainable Diagnosis for LLM Reasoning via Neuro-Symbolic Verification. *arXiv preprint arXiv:2608.08786* (2026).
- [12] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [13] Isil Dillig, Thomas Dillig, Boyang Li, and Kenneth L. McMillan. 2013. Inductive Invariant Generation via Abductive Inference. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 443–456. doi:10.1145/2491956.2462187
- [14] Ran El-Yaniv and Yair Wiener. 2010. On the Foundations of Noise-Free Selective Classification. *Journal of Machine Learning Research* 11 (2010), 1605–1641.
- [15] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *International Conference on Machine Learning*.
- [16] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [17] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.
- [18] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In *International Conference on Learning Representations*.
- [19] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1.
- [20] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*.
- [21] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations*.
- [22] Junbo Jacob Lian, Yujun Sun, Huiling Chen, Chaoyu Zhang, Hanzhang Qin, and Chung-Piaw Teo. 2026. ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization. *arXiv preprint arXiv:2602.15983* (2026).
- [23] Mark H. Liffiton and Karem A. Sakallah. 2008. Algorithms for Computing Minimal Unsatisfiable Subsets of Constraints. *Journal of Automated Reasoning* 40 (2008), 1–33. doi:10.1007/s10817-007-9084-z
- [24] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023).
- [25] Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful Chain-of-Thought Reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. 305–329. doi:10.18653/v1/2023.ijcnlp-main.20
- [26] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [27] NVIDIA. 2025. Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv:2512.20848*. doi:10.48550/arXiv.2512.20848
- [28] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. Accessed 2026-09-09. <https://openai.com/index/gpt-oss-model-card/>
- [29] Raymond Reiter. 1987. A Theory of Diagnosis from First Principles. *Artificial Intelligence* 32, 1 (1987), 57–95. doi:10.1016/0004-3702(87)90062-2
- [30] Dipankar Sarkar. 2026. From Errors to Proofs: Minimal-Core-Guided Repair for Neuro-Symbolic Constraint Solving. *arXiv preprint arXiv:2608.14771* (2026).
- [31] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*. 404–415. doi:10.1145/1168857.1168907
- [32] Tachio Terauchi and Alex Aiken. 2005. Secure Information Flow as a Safety Problem. In *Static Analysis (Lecture Notes in Computer Science, Vol. 3672)*. 352–367.
- [33] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving Math Word Problems with Process- and Outcome-Based Feedback. *arXiv preprint arXiv:2211.14275* (2022).
- [34] Shyam Upadhyay and Ming-Wei Chang. 2017. Annotating Derivations: A New Evaluation Strategy and Dataset for Algebra Word Problems. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*. Association for Computational Linguistics, 494–504. <https://aclanthology.org/E17-1047/>
- [35] Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2024. MathCoder: Seamless Code Integration in LLMs for Enhanced Mathematical Reasoning. In *International Conference on Learning Representations*.
- [36] Yuhuai Wu, Albert Qiaoqiang Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35.